InventOR: A Prompt-Configured, No-Fine-Tuning LLM Workflow for Material-Level Inventory Control

Aris Dressino[a,*]

[a] *TUM School of Management, Technical University of Munich, ArcisstraSSe 21, Munich, 80333, Germany*

Abstract

We present *InventOR*, a prompt-configured large-language-model (LLM) workflow that generates material-level (r, Q) inventory policies from cutoff-bounded historical CSVs and inline parameter blocks through an enterprise LLM application, without project-specific model training or fine-tuning. Deterministic parsing and post-cutoff simulation evaluate each LLM artifact against an SAP-derived and an SAP-safety-lead-time-informed operations-research comparator on a common 346-pair cohort from a three-plant industrial dataset. All 365 eligible plant-material pairs yield scoreable, capacity-feasible artifacts after retry handling a complete-cohort result that eliminates silent dropout. Working-day simulation reports 77.52% demand-weighted aggregate fill and 97.94% mean material fill for the LLM-emitted arm. Positive lost-sales penalty sensitivities remove the zero-penalty cost advantage. Stored metadata do not establish the exact deployed prompt or model/runtime identity; the contribution is an evaluation protocol with transparent boundary specification, manifest hashing, and prespecified exclusion rules whose first complete execution demonstrates complete-cohort

*Corresponding author. Telephone: +49 89 289 25000.
Email address: aris.dressino@tum.de (Aris Dressino)

reporting and the controls required for prospective evaluation.

Keywords: Generative Artificial Intelligence, Large Language Models, Operations Research, Supply Chain Management, Decision Support Systems, Explainable AI, Inventory Control

1. Introduction

Supply chain planning operates under volatile demand, uncertain replenishment, and disruption exposure [1]. Many material-control decisions still depend on simple parameters: reorder points, safety stock, order quantities, and review periods; these settings strongly affect service and inventory exposure [2, 3]. Classical operations research offers rigorous tools for stochastic and robust inventory decisions [4, 5, 6], but planners often need a narrower capability: converting imperfect historical records into transparent, defensible, auditable material-level policy values [7].

Large language models (LLMs) can help connect business intent, operational evidence, and analytical workflows [8, 9]. Recommendations in inventory control require explainability, reproducibility, and meaningful human control before planners can act on them [10, 11]. This creates a design requirement for hybrid AI-supported OR: LLM artifacts must be inspectable, and numeric recommendations require explicit simulation and release controls [9, 11].

This paper addresses that requirement through *InventOR*, an operations-research-first workflow for material-level inventory decision support. An enterprise LLM application produces policy artifacts from material-scoped inputs; we parse and simulate their numeric values deterministically.

Our contribution is an evaluation protocol rather than a new inventory model or solution algorithm. The comparator policies are standard textbook constructions; what we add is a reusable, auditable procedure for deciding whether LLM-emitted policy values are admissible and how they perform against ERP-derived baselines on a common simulation horizon. Three components make up that protocol.

C1 Admissibility protocol for LLM-emitted policy artifacts. Material-specific inputs pass through a configured enterprise LLM application without project-specific training or fine-tuning. Deterministic scoreability and capacity checks decide admission; the architecture reserves a planner release step, which we document as a design requirement rather than an evaluated intervention [12].

C2 Executable boundary specification. We document the material-input contract, alias-aware conversion, capacity validation, and simulation boundary, so that a competing generator can be substituted and scored under identical rules.

C3 Complete-cohort reporting discipline. All 365 eligible pairs from 233,078 parsed records yielded scoreable, capacity-feasible artifacts after retry handling; 346 pairs meet the common feasibility rules used for the three-arm descriptive comparison. No pair is silently dropped, and every exclusion rule is prespecified and reported.

The remainder of the paper is organised as follows. Section 2 reviews related work and positions InventOR relative to recent AI-supported OR contributions. Section 3 details the workflow, policy logic, and backtest design. Section 4 reports completion and simulation results. Section 5 interprets the findings and their limitations, and Section 6 summarises the contribution and outlines future work.

2. Literature Review

Operations research has long addressed supply-chain uncertainty through stochastic programming, robust optimization, and scenario-based planning [4, 5, 6]. These methods formalize variability and recourse, but operational inventory teams often face a more direct implementation problem: maintaining reorder points, safety stock, safety lead time, and order quantities from incomplete historical evidence in ERP-driven planning environments [13, 14, 15, 16, 2, 3]. Static rules can be too coarse, while sophisticated models can be difficult to explain and maintain. This motivates decision support that keeps OR discipline while making material-level parameter setting more transparent and auditable [7, 17].

Recent LLM research proposes natural-language interfaces, tool-using agents, and pipelines that translate natural-language requests into optimization model structures or executable code, including multi-agent systems that make autonomous per-period inventory decisions [9, 18, 19, 20, 21, 22]. These approaches do not specify how inventory histories should be reduced into demand regimes, lead-time diagnostics, service buffers, and feasible policy parameters with deterministic auditability.

We follow this hybrid view. We supply a filtered material CSV plus an inline parameter block through an enterprise LLM application. The intended prompt prohibits stochastic simulation, machine-learning libraries, and unrestricted data fabrication, but stored run metadata do not independently identify the deployed prompt or runtime. Because LLM outputs can contain fabricated or unfaithful content [23], our parser uses alias-aware lookups and nested-field resolution before requiring numeric positive reorder point and order quantity, numeric non-negative safety stock, and a resolvable simulator family. No project-specific model fitting or fine-tuning is performed. We treat the LLM as an artifact-generation component inside a bounded decision process. Table 1 positions our framework relative to recent AI-supported OR and inventory-control work.

Table 1: Literature positioning: InventOR relative to recent AI-supported OR contributions for inventory and supply chain decision support.

Study	**LLM role**	**Policy class**	**Demand model**	**Analytical result**	**Evaluation**
Huang et al. [18]	NL→code	General OR	Not specified	None	Benchmark
Ahmaditeshnizi et al. [20]	NL→model	General OR	Not specified	Solver-linked	Benchmark
Zhang et al. [21]	Reasoning agent	General OR	Not specified	Model and solve	Case studies
Quan and Liu [22]	Multi-agent control	Inventory	Sequential demand	Per-period decisions	Simulation
Prak et al. [24]	None	Inventory control	Compound Poisson / intermittent	Robust estimation	Empirical study
Babai et al. [25]	None	Forecasting-linked inventory	Regime-specific	Review synthesis	Review
Wasserkrug et al. [9]	Interface	General	Not specified	None	Conceptual
Cohen et al. [19]	Vision	General SCM	Not specified	None	Vision statement
InventOR	**Prototype**	(r, Q) **executed;** (s, S) **implemented, unexercised**	**Artifact-defined values**	**Descriptive**	**Post-cutoff simulation**

The table highlights the paper's narrower contribution: material-level parameter artifacts evaluated by a deterministic policy-family simulator. Unlike InvAgent's sequential multi-agent decisions, InventOR emits one stationary policy per material from a bounded historical package; unlike OptiMUS and OR-LLM-Agent, it does not formulate or solve general optimization models. The (r, Q) framework is the workhorse of the continuous-review inventory literature [13, 16]; the intermittent-demand stream represented by Prak et al. [24] builds on Croston [26] and Syntetos-Boylan [27] to handle sporadic, non-normal demand patterns which our current prototype does not explicitly model.

3. Methodology

The unit of analysis is a plant-material pair. The framework records a chain of custody from historical records to planner-facing policy values by separating deterministic input preparation, configured LLM inference, schema parsing, deterministic simulation, and human review [7, 28]. Each LLM session receives one filtered, material-scoped historical CSV as a session resource and a small inline parameter block containing current cost, MOQ, service, SAP reference, and material-level storage-capacity fields. No project-specific model weights are trained, fine-tuned, or updated. Because the stored metadata omit prompt hash, application version, and model/runtime identifier, this study does not claim that the exact deployed instruction configuration is independently reproducible. Supplementary File S1 reproduces the intended application-side system and tool prompt texts verbatim, together with their disclosed discrepancies against the executable simulator.

3.1. Problem Definition: Single-Item Continuous-Review Control with Stress-Adjusted Service Buffering

The executed task is a single-item, single-echelon continuous-review simulation with lost sales [14, 15, 29]. Each plant-material pair is treated independently; multi-item coupling, shared capacity, substitution, and production scheduling are outside the pilot. For the SAP-SLT-informed OR comparator, pre-cutoff working-day demand yields mean demand $\bar{d}_i$, annualized demand $\bar{D}_i^{\text{ann}} = 250\bar{d}_i$, lead-time-demand dispersion, and stationary

(R_i, Q_i) parameters. This comparator is not independent of ERP inputs because source SAP safety lead time enters effective lead time. For the SAP-derived arm, source safety stock is retained while R_i and Q_i are recomputed by the same executable routines. For the LLM arm, scoreable policy parameters originate directly in the artifact and pass alias-aware compatibility parsing plus strict executable-contract validation; the reference equations below do not reconstruct them. The simulator executes validated (r, Q) artifacts as fixed-quantity policies and validated (s, S) artifacts as order-up-to policies. In this study all 365 scoreable artifacts resolved to the (r, Q) family after validation, so the order-up-to path is an available but unexercised capability rather than an evaluated result.

Reference policy calculation. The following equations describe the executed SAP-SLT-informed OR comparator; they do not reconstruct direct LLM-emitted policies. The order quantity $Q_i > 0$ is rounded upward to an integer, floored at source MOQ, and then projected to its material-level storage limit when possible. The reorder point $R_i \geq 0$ triggers replenishment. The EOQ anchor minimizes the standard annual cost approximation

$$\min_{Q_i} TC_i(Q_i; SS_i) = h_i\left(\frac{Q_i}{2} + SS_i\right) + K_i\frac{\bar{D}_i^{\text{ann}}}{Q_i},$$

where SS_i is arm-specific safety stock, $h_i = \texttt{holding\_cost\_rate}_i \cdot \texttt{unit\_cost}_i$, and $K_i = \texttt{ordering\_cost}_i$. The implementation first uses $Q_i = \max\{\text{MOQ}_i, \lceil\sqrt{2K_i(250\bar{d}_i)/h_i}\rceil\}$ when demand and ordering cost are positive, and MOQ otherwise. For every arm, a supplied capacity requires $\text{MOQ}_i \leq Q_i$ and $SS_i + Q_i \leq \texttt{max\_storage\_units}_i$; comparator quantities above the residual limit are reduced, while pairs with $SS_i + \text{MOQ}_i$ above the limit are excluded from the common cohort. The SAP-SLT-informed

OR safety stock is $SS_i = \lceil z_{\alpha_i}\sigma_{LTD,i}\rceil$, with $R_i = \lceil \bar{d}_i L_i^{\text{eff}} + SS_i\rceil$. The SAP-derived arm substitutes source SAP safety stock into the same reorder-point expression. No lot-size multiple or stress coefficient is executed. The primary holding/order cost has a zero shortage penalty; positive lost-sales penalties are sensitivity analyses.

Core constraints. Inventory evolves under a lost-sales continuous-review rule on the working-day decision set $\mathcal{W}$. All arms share the first observed post-cutoff on-hand inventory; the SAP-SLT-informed OR $R_i + Q_i$ is used only when that observation is unavailable. The initial on-order pipeline is empty. Duplicate same-material, same-date rows are aggregated by summing demand, receipts, corrections, and scrap and averaging positive lead times, but the simulator consumes only demand and excludes observed receipts, corrections, and scrap. The simulator rounds mean source lead time and schedules every modeled order by that number of subsequent working observations:

$$I_{i,t} = \max(0,\ I_{i,t-1} + a_{i,t} - d_{i,t}), \quad t \in \mathcal{W},$$

where $a_{i,t}$ is the quantity whose working-day due observation is no later than t. Scheduled arrivals are added first, realized demand is served subject to lost sales, and the post-demand inventory position is computed as on-hand plus outstanding orders. For a validated (r, Q) artifact, a fixed quantity is ordered when

$$\delta_{i,t} = \mathbf{1}\{\mathrm{IP}^+_{i,t} \le R_i\}, \qquad \mathrm{IP}^+_{i,t} = I_{i,t} + \sum_{s:\,\mathrm{due}(s)>t} o_{i,s},$$

with $o_{i,t} = Q_i\delta_{i,t}$. For a validated (s, S) artifact, the same threshold test uses s_i, and an order of $S_i - \mathrm{IP}^+_{i,t}$ is placed when positive; no artifact in this study triggered that branch. Orders due after the observed horizon remain in transit rather than being forced onto the final observation. Parameters remain stationary. Shared observed initialization and the empty initial pipeline are part of the executed comparison design.

The reorder point targets a nominal cycle service level $\alpha_i \in [0.5, 0.999]$ after numeric clamping. Under the normal approximation,

$$\mathbb{P}\Big(D_{i,L_i^{\text{eff}}} \leq R_i\Big) \geq \alpha_i \iff R_i \geq \bar{d}_i L_i^{\text{eff}} + z_{\alpha_i}\, \sigma_{LTD,i}.$$

The reported backtest metric is the ex-post fill rate, which is distinct from the nominal cycle-service target because it also depends on Q_i and realized demand [30].

Lead-time-demand variance is propagated under the demand and lead-time independence approximation,

$$\sigma_{LTD,i}^2 = L_i^{\text{eff}} \sigma_{D,i}^2 + \bar{d}_i^2 \sigma_{L,i}^2.$$

The executed SAP-SLT-informed OR buffer is

$$SS_i^{\text{OR}} = z_{\alpha_i} \sigma_{LTD,i},$$

with values rounded upward:

$$SS_i^{\text{OR}} = \lceil z_{\alpha_i} \sigma_{LTD,i} \rceil, \qquad R_i = \left\lceil \bar{d}_i L_i^{\text{eff}} + SS_i^{\text{OR}} \right\rceil.$$

The executable statistics add non-negative source SAP safety-lead-time days to mean lead time when constructing comparator lead-time-demand dispersion (Section 3.5 discusses this confound). LLM-emitted values remain artifact values accepted by alias-aware parsing and strict executable-contract validation.

Parser and review boundary. Artifact policy labels determine simulator routing after validation. The parser resolves compatibility aliases, then requires a valid (r, Q) tuple or explicit s and S values for an (s, S) label; it rejects malformed, infeasible, or internally inconsistent artifacts. It does not execute label-specific optimization or deterministic escalation. Because no artifact supplied explicit and internally consistent s and S values, every scored policy in this study was executed as (r, Q); the reported evidence therefore concerns fixed-quantity control only. Planner review remains required before any operational use.

3.2. Historical Input Schema and Canonical Analysis Table

The uploaded CSV contains only the subset of source-table columns listed in Table 2. Plant and material identifiers are carried by the run key and inline block rather than repeated in the uploaded rows. The inline block passes cost, MOQ, service, SAP reference, and material-level storage-capacity variables. Stock-flow fields omitted from the uploaded subset are not analyzed by the executed LLM path.

Table 2: Fields included in the material-scoped CSV uploaded by the executed runner.

Column	Type	Description and method use
`date`	date / datetime	Calendar observation date; time ordering, aggregation, and diagnostics
`actual_demand`	numeric	Realized demand; mean, variability, intermittency, trend, and regime classification
`delivery_time`	numeric	Recorded replenishment lead time; rounded and scheduled as subsequent working observations by the simulator
`minimum_order_quantity`	numeric	Minimum replenishment quantity; order-size feasibility constraint
`safety_stock_units`	numeric	Existing safety stock; benchmarking against computed recommendations
`safety_time_workdays_actual`	numeric	Source SAP safety-lead-time field; currently enters comparator effective lead time
`supplier_plant_distance`	numeric	Supplier-to-plant distance; contextual lead-time-risk information
`reliability_calc`	numeric	Source supplier or lane reliability context
`transportation_network`	string	Source transportation-network context
`service_level_target`	numeric	Source service target used by comparator calculations
`unit_cost`	numeric	Purchase or standard cost per unit in the source dataset's internal cost denomination; inventory-value proxy and input to holding-cost derivation and EOQ calculation
`holding_cost_rate`	numeric	Annual carrying-cost rate as a decimal (e.g., 0.20 for 20% of unit value); used to derive the annual cost of carrying one unit in stock, balance replenishment frequency against stock exposure, and compute economic order quantity
`ordering_cost`	numeric	Fixed replenishment cost in the source dataset's internal cost denomination; input to EOQ calculation and simulated material-horizon holding/order cost
`max_storage_units`	numeric	Material-level upper bound applied to all evaluated arms: MOQ $\leq Q$ and $SS + Q \leq$ `max_storage_units`
`is_working_day`	binary	Working-day flag; working-day demand statistics and non-zero demand share

3.3. Analytical Workflow

The workflow transforms a material snapshot into a policy artifact through preparation, LLM inference, parsing, and simulation (Figure 1).

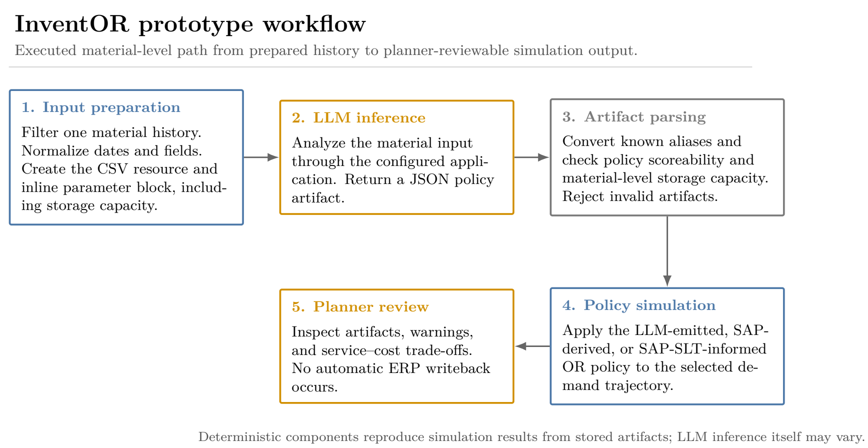

Figure 1: Prototype workflow from material input preparation to policy artifact and simulation output.

The runner uploads a normalized CSV and inline block; the application returns a JSON artifact. The parser converts it to an executable (r, Q) or (s, S) policy and rejects malformed outputs. Missing artifacts are retried. The simulator evaluates LLM, SAP-derived, and SAP-SLT-informed OR policies on the common horizon. An aggregate-only manifest hashes active-input, evaluation-code, and stored-artifact files; it records unavailable provenance fields explicitly. Current runner code records these fields for future runs when the corresponding environment variables are supplied. The author declares retrospectively that evaluated artifacts were produced with an Anthropic Claude Sonnet 4.6 model behind the enterprise application, using a code-interpreter-style Python execution tool; this is recorded as unverified because no deployed model identifier or generation settings were persisted, and must not be read as machine-verified provenance. Stored inputs, artifacts, and code reproduce a simulation, not a fresh LLM completion; planners retain implementation authority.

3.4. Artifact Completeness and Future Readiness

The completeness audit asks whether each eligible pair has a scoreable artifact after retries; it does not estimate first-pass reliability, explanation completeness, or deployment readiness. The empirical section is a retrospective audit of an already-executed workflow rather than a controlled experiment: the generation cohort was produced before the evaluation protocol was finalized, the deployed configuration was not frozen or instrumented in advance, and no treatment was assigned. The post-cutoff simulation uses artifacts generated exclusively from pre-cutoff inputs and is descriptive rather than a controlled prospective trial [31, 32]. A future evaluation must freeze the deployed configuration, model/runtime identifiers, access controls, and approval workflow.

3.5. Safety Lead Time Computation

Safety lead time (SLT) is an ancillary ERP-facing design field. The prototype specification proposes combining transport, supplier, and quality lead-time variability with a reliability-deficit premium [14, 3], but the present release does not document estimators for those components. In the executed comparator code, non-negative source SAP safety-lead-time days are added to mean lead time before computing both SAP-derived and OR-computed dispersion and reorder points (this coupling is also flagged in Section 3.1). The OR arm is therefore labelled SAP-SLT-informed, and no separate SLT effect is identified.

3.6. Post-Cutoff Policy Simulation

The confidential operational extract contains 233,078 parsed CSV records, 411 plant-material pairs, and three anonymized plants (Plant

A, Plant B, and Plant C). The parameter cutoff is 2026-03-10 and the post-cutoff simulation horizon ends on 2026-08-27. Eligibility filters retain 365 plant-material pairs with sufficient pre-cutoff and post-cutoff evidence: 234 from Plant A, 65 from Plant B, and 66 from Plant C. Plant C has no positive source working-day flags, so its calendar flags are derived from same-date Plant A and Plant B votes; ties classify as working days. This deterministic repair permits a common decision calendar; because no alternative Plant C calendar exists, robustness is examined by reporting calendar-day arrivals and by re-running the full comparison on the native-calendar subset that excludes Plant C entirely. SAP-derived and SAP-SLT-informed OR parameters use pre-cutoff information. Stored LLM artifacts were generated from cutoff-bounded pre-cutoff inputs, so every arm is evaluated on the same post-cutoff horizon. Nineteen pairs cannot meet comparator safety stock plus MOQ within their supplied capacity; these are excluded before the common 346-pair comparison.

The simulated arms are: (i) SAP-derived, retaining source SAP safety stock while recomputing reorder point and EOQ order quantity from pre-cutoff statistics; (ii) SAP-SLT-informed OR, using the same statistics, source safety-lead-time input, and feasibility rules but without LLM inference; and (iii) LLM-emitted, a validated (r, Q) or (s, S) artifact. The simulator is continuous-review and lost-sales: all arms begin with the same first observed post-cutoff on-hand inventory, using the OR $ROP + Q$ only as a shared fall-back. The initial on-order pipeline is empty. On each working row, modeled arrivals due after rounded mean working-day lead time are received, demand is served subject to lost sales, post-demand inventory position is checked, and a policy-specific order is placed if its threshold is met. Observed post-cutoff receipts, corrections, and scrap are excluded. This common startup design may create cutoff transients. Metrics are fill rate, aggregate demand-weighted fill rate, materials above 95% fill, zero-stockout materials, stockout days, average on-hand inventory, and simulated material-horizon cost.

For policy p and material i, the cost summaries use the simulated horizon of T_i working days. Let $\overline{I}_{i,p}$ be average on-hand inventory, h_i annual holding cost per unit, K_i fixed ordering cost, and $n_{i,p}^{\text{orders}}$ simulated orders. The implementation uses a constant 250 annual working days and computes

$$TC_{i,p} = h_i \overline{I}_{i,p} \frac{T_i}{250} + K_i n_{i,p}^{\text{orders}}.$$

This is a simulated material-horizon holding/order cost in the source dataset's internal cost denomination, not an audited annual financial or booked accounting result. Table 3 reports arithmetic means across the cohort with zero shortage penalty. Sensitivities add lost-sales penalties of one and five times source unit cost; cost values must be read jointly with fill rate and stockout days.

4. Results

4.1. Completion and Artifact Validity

The final execution produced parser-scoreable artifacts for all 365 eligible plant-material pairs with zero missing and zero parser-rejected outputs after retry handling. Every artifact passes alias-aware parsing, nested-field resolution, and capacity validation against the supplied material-level storage bound; all resolve to the (r, Q) family after validation.

4.2. Main Three-Arm Exploratory Simulation

Table 3 reports three policy arms on the same 346 common-feasible material trajectories under working-day post-cutoff simulation. All arms satisfy identical MOQ and supplied per-material storage rules; 19 otherwise eligible pairs whose comparator safety stock plus MOQ exceed capacity are excluded.

Table 3: Post-cutoff working-day simulation on 346 common-feasible plant-material pairs. All arms use pre-cutoff parameterization or cutoff-bounded inputs and satisfy identical MOQ and supplied storage rules. Cost is mean simulated material-horizon holding/order cost with zero shortage penalty. SO days is the total count of stockout days across the cohort.

Policy	n	**Mean FR (%)**	**Agg. FR (%)**	**≥95% (%)**	**Zero SO (%)**	**Mean cost**	**SO days**
SAP-derived	346	98.38	79.99	94.51	80.35	3,184.20	741
SAP-SLT OR	346	98.27	79.58	93.93	88.73	3,852.22	799
LLM-emitted	346	97.94	77.52	92.49	81.50	2,848.96	936

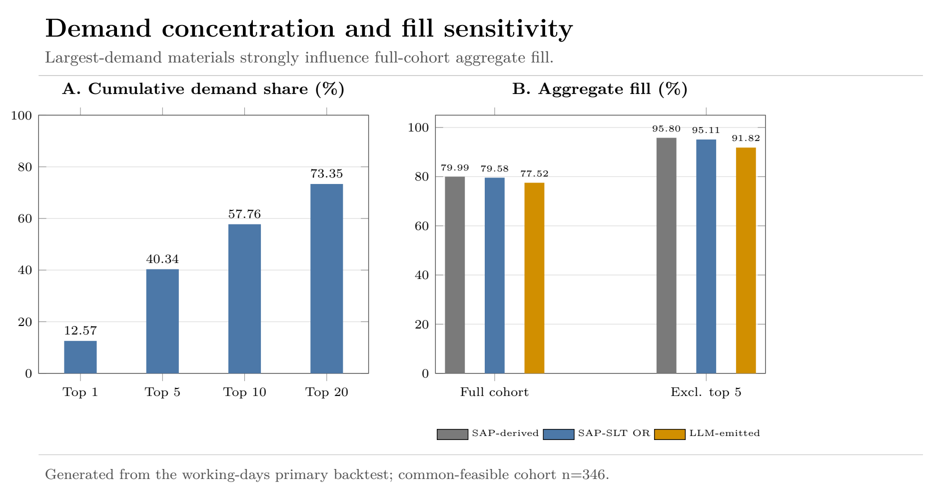

Figure 2: Demand concentration and aggregate-fill sensitivity. Panel A reports cumulative demand share for the largest materials. Panel B compares full-cohort aggregate fill with the value after excluding the five largest-demand materials.

Relative to SAP-derived, LLM-emitted policies show 2.48 percentage points lower aggregate fill, 335.24 lower mean holding/order cost, and 195 additional stockout days. Relative to SAP-SLT-informed OR, 2.07 percentage points lower aggregate fill, 1,003.26 lower mean holding/order cost, and 137 additional stockout days.

Aggregate evidence is demand-concentrated. The largest 1, 5, 10, and 20 materials contribute 12.57%, 40.34%, 57.76%, and 73.35% of post-cutoff demand. Excluding the five largest-demand materials raises aggregate fill to 91.82% for LLM-emitted, 95.80% for SAP-derived, and 95.11% for SAP-SLT-informed OR, versus 77.52–79.99% in the full cohort (Figure 2). The largest-demand items drive aggregate fill.

At material level, LLM-emitted fill is not lower than SAP-derived for 314 of 346 materials (90.8%), while its cost is no higher for 238 (68.8%); both conditions hold for 211 (61.0%). Against SAP-SLT-informed OR, the corresponding counts are 309 (89.3%), 303 (87.6%), and 269 (77.7%). Paired LLM-minus-comparator fill differences have first quartile, median, and third quartile of 0.00 percentage points for both comparators, reflecting many exact fill ties. Corresponding cost differences are -259.24, -47.17, and 34.96 versus SAP-derived and -787.83, -209.32, and -25.26 versus SAP-SLT-informed OR. A deterministic 10,000-resample paired bootstrap gives LLM-minus-SAP fill and cost means of -0.44 percentage points (95% interval $[-0.99, -0.02]$) and -335.24 (95% interval $[-629.16, -105.29]$); against SAP-SLT-informed OR, the corresponding values are -0.33 percentage points ($[-0.89, 0.14]$) and $-1{,}003.26$ ($[-1{,}363.81, -705.03]$). These descriptive resampling intervals and aggregate results show service–cost trade-offs rather than uniform policy ranking.

The artifact audit covers all 365 scoreable policies: normalized policy-family agreement is 90.68%, and 42 artifacts meet a prespecified diagnostic-outlier rule (CV absolute error above 0.75 or LLM-to-pipeline safety-stock ratio outside [0.5, 2.0]). Removing the 23 flagged artifacts present in the common cohort raises LLM aggregate fill from 77.52% to 79.55%, while SAP-derived reaches 80.08% and SAP-SLT-informed OR reaches 79.70%; this post-hoc diagnostic subset does not establish a causal correction. Replacing primary working-day receipt offsets with calendar-day offsets gives aggregate fills of 77.77%, 80.25%, and 79.84%, respectively. Removing the one plant whose source working-day flags are entirely absent, and whose calendar is therefore imputed, leaves 280 native-calendar pairs with aggregate fills of 74.03% for LLM-emitted, 76.55% for SAP-derived, and 76.35% for SAP-SLT-informed OR, mean material-level fills of 98.19%, 98.71%, and 98.54%, zero-stockout shares of 81.79%, 88.21%, and 90.71%, and mean holding/order costs of 2,893.68, 3,471.66, and 4,168.76. In this native-calendar subset both comparators retain higher fill and higher zero-stockout shares than the LLM arm, so the imputed calendar does not manufacture the observed service gap. With a lost-sales penalty of one times unit cost, mean total costs are 1,688,922.75, 1,631,321.28, and 1,692,746.91, respectively; at five times unit cost, they are 8,433,217.92, 8,143,869.56, and 8,448,325.64. Thus, zero-penalty holding/order cost cannot establish economic superiority. These sensitivities do not resolve empty-pipeline, excluded-flow, or calendar-repair limitations.

5. Discussion

We identify three findings as the principal scientific contribution. First, 100% of 365 eligible plant-material pairs yield scoreable, capacity-feasible policy artifacts after retry handling a complete-cohort result that eliminates silent dropout and establishes a benchmark for future LLM-based inventory studies. Second, all scored artifacts resolve to the (r, Q) family; the LLM constructs every policy value from only the material-scoped history and parameter block, while both comparators are warm-started from ERP settings that embed accumulated planner judgment. The observed aggregate-fill deficit of roughly two percentage points, and the higher stockout-day count (936 versus 741 for SAP-derived and 799 for SAP-SLT-informed OR), are therefore cold-start characteristics of a generator that receives no safety-stock legacy or safety-lead-time input. An ERP-independent cold-start OR arm is the comparator required to isolate generator quality and is specified as a future-work requirement. Third, positive lost-sales penalty sensitivities remove the zero-penalty cost ranking, confirming that the apparent cost advantage is driven by the penalty assumption rather than by generator quality.

The evaluation protocol is the reusable contribution. An aggregate-only manifest hashes active inputs, evaluation code, and stored artifacts, flagging unavailable provenance fields. The input-contract specification, alias-aware parser, capacity-validation rules, and the full exclusion log form a reproducible boundary within which any LLM or OR generator can be substituted and scored under identical rules. The tracked historical 315-material snapshots are byte-identical rather than distinct generation replicates [33]; repeat inference at cohort scale with fixed instrumentation belongs among future-work requirements.

Our study is bounded by a single anonymized three-plant dataset, one generation configuration, and a post-cutoff simulation that excludes observed receipts, corrections, and scrap with an empty initial pipeline. These are explicit design choices, not undisclosed weaknesses. The contribution is the protocol and its first complete execution, not a performance claim.

6. Conclusion

We present *InventOR*, an applied AI/OR prototype that treats LLMs as candidate-policy generators within deterministic preparation, parsing, and simulation. The framework preserves an inspectable link between material histories, policy fields, and simulated outcomes without project-specific model training or fine-tuning.

The executed artifact set contains 365 scoreable policies generated from cutoff-bounded, pre-cutoff material histories, all satisfying their material-level storage bounds. On the 346-pair cohort where all arms satisfy identical MOQ and supplied storage rules, LLM-emitted aggregate fill is 77.52%, mean material-level fill is 97.94%, mean simulated material-horizon holding/order cost is 2,848.96, and stockout days total 936 under working-day receipt scheduling. The warm-start asymmetry explains the fill deficit and higher stockout-day count relative to ERP-informed comparators. Positive shortage-cost sensitivities remove the zero-penalty cost advantage. These values are descriptive simulated-comparison evidence, not deployment or production-trial evidence.

Future work should supply an ERP-independent cold-start OR arm, repeat inference at cohort scale to estimate generation variance, extend the admissible policy family beyond (r, Q) to order-up-to and intermittent-demand regimes, and run a prospective evaluation with planner-in-the-loop review and fixed instrumentation. An ex-ante diagnostic-outlier exclusion rule and a documented SLT-component estimator belong in the same evaluation protocol upgrade.

Author Contributions

Conceptualization, Methodology, Software, Formal analysis, Validation, Visualization, Writing – original draft, Writing – review and editing: Aris Dressino.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data and Code Availability

The raw operational data and material-level artifacts cannot be publicly released because they contain commercially sensitive material, supplier, and planning information. A confidentiality-screened release package containing analysis code, the anonymized variable schema, aggregate evaluation results, manuscript sources, and aggregate integrity manifest is available at `github.com/DressPD/inventor_public`. The release package is additionally deposited in a Zenodo archive whose version-specific DOI will be registered at acceptance and cited in the final version of record; the GitHub repository is the development mirror and the Zenodo deposit is the citable archival copy. Supplementary File S1 accompanies this submission and contains the intended application-side prompt specification. The historical deployed application configuration, model/runtime identifier, inference dates, and prompt hashes were not preserved and are not claimed as available; the current runner records these fields for future executions.

7. Acknowledgments

The author thanks the industrial collaborators and material-control experts who contributed domain knowledge and access to anonymized operational data. The author also acknowledges the associated enterprise data and IT teams for supporting data access and prototype execution. No formal full-population planner evaluation is claimed. Any errors or omissions remain the responsibility of the author.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During manuscript preparation, the author used OpenAI GPT-5.6 through OpenCode for language editing, consistency checks, citation review, and LaTeX proofing support; this is a writing-assistance tool and is distinct from the Anthropic Claude Sonnet 4.6 deployment evaluated in the research workflow. The author reviewed and edited the content after use and takes full responsibility for the content of the published article. Figures were rendered deterministically from author-reviewed LaTeX and Python source; generative AI did not directly create or manipulate image content. The research workflow evaluated in the paper is itself the object of study and is described separately in the methodology.

References

[1] S. Sengupta, P. Jonsson, H. Dreyer, R. Kaipia, T. Y. Choi, Uncertainty regulation and adaptable supply chain planning, International Journal of Operations & Production Management 45 (11) (2025) 1861–1883.

[2] E. T. Grasso, B. W. Taylor, A simulation-based experimental investigation of supply/timing uncertainty in MRP systems, International Journal of Production Research 22 (3) (1984) 485–497. doi:10.1080/00207548408942468.

[3] T. J. van Kampen, D. P. van Donk, D.-J. van der Zee, Safety stock or safety lead time: coping with unreliability in demand and supply, International Journal of Production Research 48 (24) (2010) 7463–7481. doi:10.1080/00207540903348346.

[4] D. Bertsimas, A. Thiele, A robust optimization approach to inventory theory, Operations Research 54 (1) (2006) 150–168. doi:10.1287/opre.1050.0238.

[5] A. Thorsen, T. Yao, Robust inventory control under demand and lead time uncertainty, Annals of Operations Research 257 (1) (2017) 207–236.

[6] D. Zhang, H. H. Turan, R. Sarker, D. Essam, Robust optimization approaches in inventory management: Part A—the survey, IISE Transactions 57 (7) (2025) 818–844.

[7] A. J. Mersereau, Information-sensitive replenishment when inventory records are inaccurate, Production and Operations Management 22 (4) (2013) 792–810.

[8] T. Boone, B. Fahimnia, R. Ganeshan, D. M. Herold, N. R. Sanders, Generative AI: Opportunities, challenges, and research directions for supply chain resilience, Transportation Research Part E: Logistics and Transportation Review 199 (2025) 104135.

[9] S. Wasserkrug, L. Boussioux, D. Den Hertog, F. Mirzazadeh, Ş. İ. Birbil, J. Kurtz, D. Maragno, Enhancing decision making through the integration of large language models and operations research optimization, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 39, 2025, pp. 28643–28650.

[10] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, et al., Guidelines for human-AI interaction, in: Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, 2019, pp. 1–13.

[11] I. D. Raji, A. Smart, R. N. White, M. Mitchell, T. Gebru, B. Hutchinson, J. Smith-Loud, D. Theron, P. Barnes, Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing, in: Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, 2020, pp. 33–44. doi:10.1145/3351095.3372873.

[12] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, A. Chadha, A systematic survey of prompt engineering in large language models: Techniques and applications, arXiv preprint arXiv:2402.07927 (2024). doi:10.48550/arXiv.2402.07927.

[13] G. Hadley, T. M. Whitin, Analysis of Inventory Systems, Prentice-Hall, Englewood Cliffs, NJ, 1963.

[14] E. A. Silver, D. F. Pyke, D. J. Thomas, Inventory and Production Management in Supply Chains, 4th Edition, CRC Press, Boca Raton, FL, 2017.

[15] P. H. Zipkin, Foundations of Inventory Management, McGraw-Hill, New York, 2000.

[16] E. L. Porteus, Foundations of Stochastic Inventory Theory, Stanford University Press, Stanford, CA, 2002.

[17] Y. Kang, S. B. Gershwin, Information inaccuracy in inventory systems: stock loss and stockout, IIE transactions 37 (9) (2005) 843–859.

[18] C. Huang, Z. Tang, S. Hu, R. Jiang, X. Zheng, D. Ge, B. Wang, Z. Wang, ORLM: A customizable framework in training large models for automated optimization modeling, Operations Research 73 (6) (2025) 2986–3009. doi:10.1287/opre.2024.1233.

[19] M. C. Cohen, T. Dai, G. Perakis, N. Agrawal, G. Allon, R. N. Boute, G. P. Cachon, Z. Chen, M. Cohen, R. Cristian, et al., OM Forum—Supply Chain Management in the AI Era: A vision statement from the operations management community, Manufacturing & Service Operations Management 28 (3) (2026) 687–705. doi:10.1287/msom.2025.1065.

[20] A. Ahmaditeshnizi, W. Gao, M. Udell, OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models, in: Proceedings of the 41st International Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, PMLR, 2024, pp. 577–596.
URL `https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html`

[21] B. Zhang, P. Luo, G. Yang, B.-H. Soong, C. Yuen, OR-LLM-Agent: Automating modeling and solving of operations research optimization problems with reasoning LLM, arXiv preprint arXiv:2503.10009 (2025). doi:10.48550/arXiv.2503.10009.

[22] Y. Quan, Z. Liu, InvAgent: A large language model based multi-agent system for inventory management in supply chains, arXiv preprint arXiv:2407.11384 (2024). doi:10.48550/arXiv.2407.11384.

[23] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, P. Fung, Survey of hallucination in natural language generation, ACM Computing Surveys 55 (12) (2023) 1–38. doi:10.1145/3571730.

[24] D. Prak, R. Teunter, M. Z. Babai, J. E. Boylan, A. Syntetos, Robust compound Poisson parameter estimation for inventory control, Omega 104 (2021) 102481. doi:10.1016/j.omega.2021.102481.

[25] M. Z. Babai, A. A. Syntetos, R. H. Teunter, Fifty years of inventory research from a forecasting perspective, European Journal of Operational Research 331 (1) (2026) 1–20. doi:10.1016/j.ejor.2025.07.003.

[26] J. D. Croston, Forecasting and stock control for intermittent demands, Operational Research Quarterly 23 (3) (1972) 289–303. doi:10.1057/jors.1972.50.

[27] A. A. Syntetos, J. E. Boylan, The accuracy of intermittent demand estimates, International Journal of Forecasting 21 (2) (2005) 303–314. doi:10.1016/j.ijforecast.2004.10.001.

[28] S. Watts, G. Shankaranarayanan, A. Even, Data quality assessment in context: A cognitive perspective, Decision Support Systems 48 (1) (2009) 202–211.

[29] M. Bijvank, I. F. A. Vis, Lost-sales inventory theory: A review, European Journal of Operational Research 215 (1) (2011) 1–13. doi:10.1016/j.ejor.2011.02.004.

[30] S. Axsäter, Inventory Control, 3rd Edition, Vol. 225 of International Series in Operations Research & Management Science, Springer, Cham, 2015.

[31] L. J. Tashman, Out-of-sample tests of forecasting accuracy: an analysis and review, International Journal of Forecasting 16 (4) (2000) 437–450. doi:10.1016/S0169-2070(00)00065-0.

[32] H. Hewamalage, K. Ackermann, C. Bergmeir, Forecast evaluation for data scientists: common pitfalls and best practices, Data Mining and Knowledge Discovery 37 (2) (2023) 788–832. doi:10.1007/s10618-022-00894-5.

[33] S. Ouyang, J. M. Zhang, M. Harman, M. Wang, An empirical study of the non-determinism of ChatGPT in code generation, ACM Transactions on Software Engineering and Methodology 34 (2) (2025) 1–28. doi:10.1145/3697010.